

RTL Design Sherpa

GATLING-RV32ITETRAD Hardware Architecture Specification 0.1

October 22, 2026

Table of Contents

1 Gatling Has Index....7

2 Document Information....8

2.1 GATLING-RV32ITETRAD Hardware Architecture Specification....8

2.2 Document Purpose....9

2.3 References....10

2.4 Revision History....12

3 Introduction....13

3.0.1 Figure 1.1: gatling in a system....14

4 Programmer's Model....15

4.1 Contexts....15

4.1.1 Figure 2.1: What a program sees....15

4.2 ISA....16

4.3 CSRs....17

4.4 Halts....18

4.5 Timing a program can rely on....19

5 Interfaces....20

5.1 Core memory ports....20

5.2 Board memory map (gatling_mem_bram, over UART -> AXI4-Lite)....21

5.2.1 Figure 3.1: The board address map....21

6 Performance....23

7 GATLING-RV32ITETRAD Micro-Architecture Specification Index....24

7.1 Document Organization....25

7.1.1 Front Matter....25
7.1.2 Chapter 1: Overview....25
7.1.3 Chapter 2: Functional Blocks....25
7.1.4 Chapter 3: Internal Interfaces....25
7.1.5 Chapter 4: Micro-Architecture Behavior....25
7.1.6 Chapter 5: Verification....25
7.1.7 Chapter 6: Configuration....25
7.1.8 Chapter 7: Board Result....25
7.2 Related Documentation....26
8 The Barrel Model: Why Four Contexts....27
8.1 1. The cost of one cycle....28
8.2 2. The obvious fix, and why it does not pay for one stream....29
8.3 3. The interleave....30
8.4 4. What the four stages buy in memory....31
8.5 5. What it costs....32
8.6 6. Measured....33
8.7 7. Sources....34
9 GATLING PRD — Tetrad Compute Engine....35
9.1 1. Purpose....36
9.2 2. Non-goals....37
9.3 3. ISA — RV32I + Xcell....38
9.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)....38
9.3.2 3.2 Assembly mnemonics....40
9.3.3 3.3 CSRs, and the deliberate departures from the RISC-V spec....40

9.4 4. Barrel microarchitecture....42
9.4.1 4.1 The model....42
9.4.2 4.2 Structural hazards (all resolved by rotation, by construction)....42
9.4.3 4.3 Branch behavior....42
9.4.4 4.4 Halt/trap....42
9.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)....43
9.6 6. Verification....44
9.7 7. Open items (owner decisions needed)....45
9.8 8. Deliverables & sequencing....46
9.9 9. History....47
10 Microarchitecture Reading List....48
10.1 Foundations — kestrel & merlin....49
10.2 Prediction, traps, caches — peregrine....50
10.3 Out-of-order — gyrfalcon....51
10.4 ISA rationale & prior art — all rungs....52
10.5 Interleaved multithreading — gatling....53
10.6 Textbooks....54

List of Figures

Figure 1.1: gatling in a system.....14
Figure 2.1: What a program sees.....15
Figure 3.1: The board address map.....21

List of Tables

Table 1: Reference Documents....10
Table 2: Document Revision History....12
Table 3: Table 2.1: Halt causes (kestrel's encoding)....18
Table 4: Table 3.1: gatling_core interfaces....20
Table 5: Table 3.2: Board memory map....21
Table 6: Table 4.1: Throughput....23
Table 7: Table 1.1: kestrel's single cycle on the Nexys A7....28
Table 8: Table 4.1: every memory latency is exactly one stage....31
Table 9: Table 6.1: The single cycle against the barrel....33
Table 10: Table 7.1: Sources cited in this chapter....34
Table 11: Textbooks cited by the rung books....54

1 Gatling Has Index

Generated: 2026-10-22

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 GATLING-RV32ITETRAD Hardware Architecture Specification

Document Number: GATLING-HAS-001 **Version:** 0.1 **Status:** Draft **Classification:** Open Source - MIT License

---

2.2 Document Purpose

This Hardware Architecture Specification (HAS) is gatling's external contract: what a program running on a context can rely on, how the core connects to memory, and how a host loads, runs and observes the four contexts on the board. How it is built is the companion GATLING MAS.

2.3 References

Reference Documents

ID	Document	Description
[REF-1]	GATLING MAS (`docs/gatling_mas/`)	Micro-Architecture Specification (companion)
[REF-2]	GATLING PRD v0.3 (`PRD.md`)	Requirements, the Xcell encoding table and the owner decisions
[REF-3]	The Barrel Model (`docs/barrel_model/barrel_model.md`)	Study chapter: why four contexts, with the kestrel-against-gatling measurement
[REF-4]	RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (`projects/components/riscv-ip/references/riscv-spec.pdf`)	ISA authority; cited by section as “unpriv §N”
[REF-5]	RISC-V Instruction Set Manual, Volume II: Privileged Architecture (same PDF)	The CSRs and instructions gatling deliberately departs from; cited as “priv §N”
[REF-6]	riscv-formal `rvfi.md` (`vendor/riscv-formal/docs/rvfi.md`)	RVFI retire-channel field semantics
[REF-7]	KESTREL HAS (`../kestrel-rv32i/docs/kestrel_has/`)	The ISA contract gatling inherits: halt causes, the CSR stub, FENCE as a no-op
[REF-8]	J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019	Fine-grained multithreading, §3.11; the lineage from the CDC 6600 I/O processors through HEP, Tera and Niagara, Appendix M.5; cited as “H&P6 §N”
[REF-9]	D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, RISC-V Edition*, 2nd ed.,	Pipeline hazards, §4.8-4.9; hardware multithreading, §6.4; cited as “P&H §N”

ID	Document	Description
	Morgan Kaufmann, 2021	
[REF-10]	S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022	Multithreading as replicated architectural state, §7.7.7; cited as "H&H §N"
[REF-11]	Microarchitecture reading list (`projects/components/riscv-ip/references/microarchitecture-papers.md`), entries 16-20	The barrel's primary sources: Thornton (CDC 6600), Smith and Jordan (HEP), Alverson et al. (Tera), Kongetira et al. (Niagara); cited as "(papers, entry N)"

2.4 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-22	seang	Initial HAS: the RV32I barrel with `NUM_HARTS` contexts (four by default, the tetrad), the CSR stub and the deliberate departures from the RISC-V spec (PRD 3.3), the board memory map, the figures (system context, programmer's view, address map) and the references. The ISA blocks are `riscv-ip/common-rv32i`'s, shared with kestrel. Xcell (PRD 3.1) reserved, not yet built.

Last Updated: 2026-10-22

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

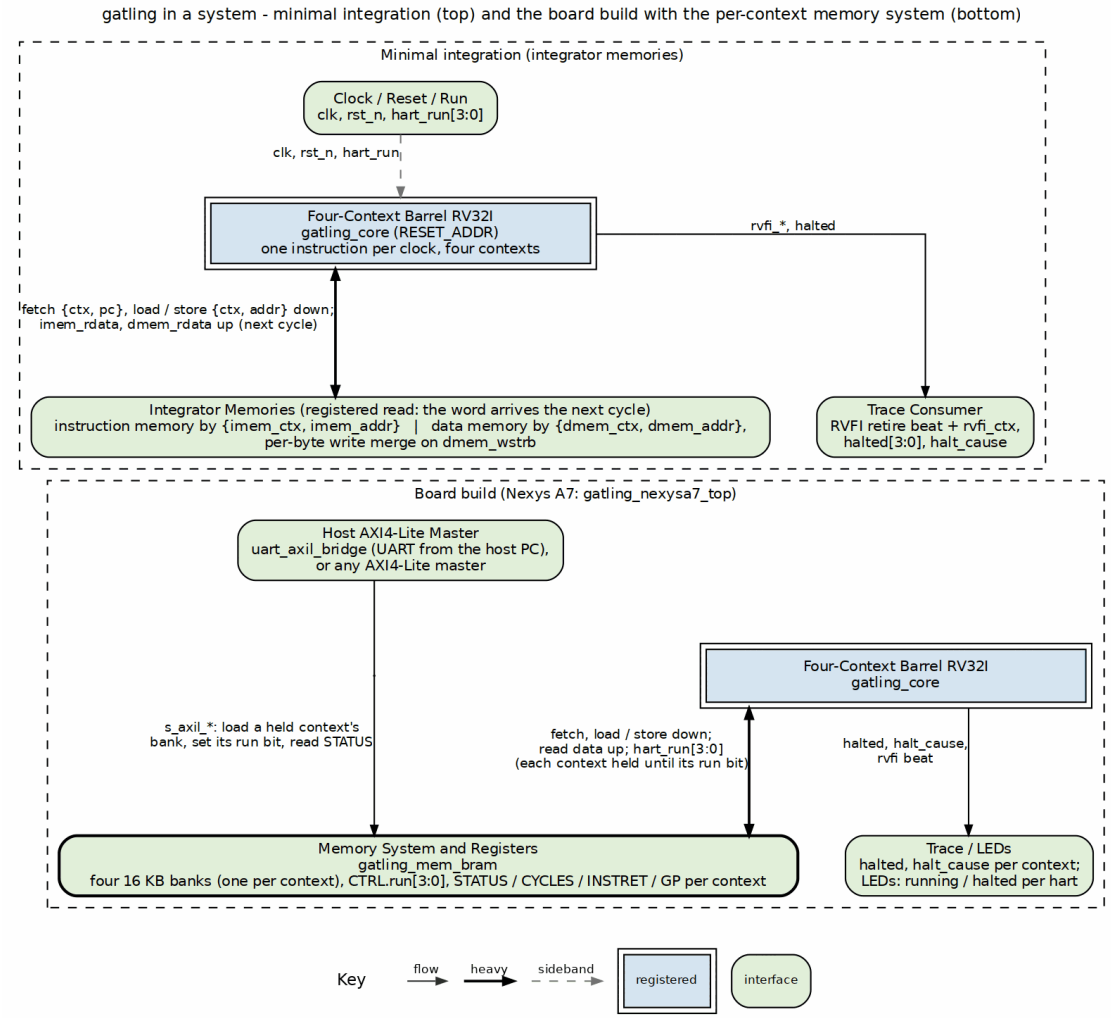
3 Introduction

gatling is the per-tetrad compute engine of the tile (PRD 1): one RV32I processor with four hardware contexts, one per cell of the tetrad, sharing one datapath. The four contexts run four independent programs. Each context executes one instruction per four clocks; together they execute one per clock.

The organization is the oldest form of hardware multithreading. Thornton's CDC 6600 ran its ten peripheral processors through one shared execution unit in strict rotation (papers, entry 16; H&P6 App. M.5), the Denelcor HEP let a process issue only when its previous instruction had left the pipeline (papers, entry 17), and the textbooks name the scheme fine-grained multithreading: switch threads every cycle, round-robin (H&P6 §3.11; P&H §6.4; H&H §7.7.7). The textbooks present the interleave as a way to hide stalls. gatling fixes the thread count equal to the pipeline depth, and at that point the hazards are not hidden but gone; the barrel model [REF-3] works through the argument with measurements.

This first build is the RV32I barrel (owner, 2026-10-21). The Xcell extension – the custom-0 instructions that move data between a context and its cell (PRD 3.1) – is reserved and arrives with the cell seam. On the Nexys A7 the four contexts run the riscv-tests rv32ui battery at once, four different programs side by side.

3.0.1 Figure 1.1: gatling in a system



gatling system context: minimal integration and the board build

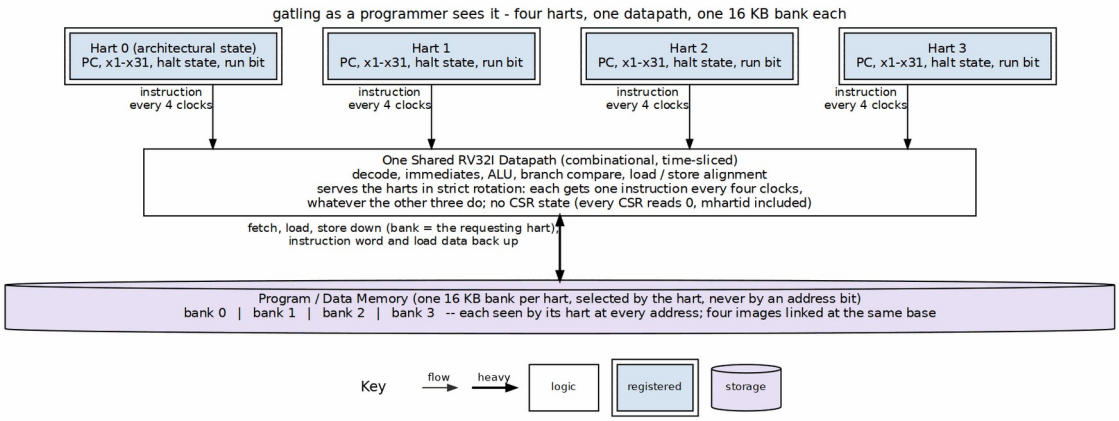
The top region is the minimal integration: `gatling_core` with two integrator-provided registered-read memories, addressed by context and address, and a trace consumer on the RVFI port. The bottom region is the board build: `gatling_mem_bram` supplies one 16 KB bank per context, the per-context run bits and the status registers, and a host (the UART-to-AXI4-Lite bridge on the Nexys A7, or any AXI4-Lite master) loads a held context's bank and releases it, while the other contexts keep running.

4 Programmer's Model

4.1 Contexts

Each context is a complete RV32I hart: a PC, registers x0-x31 and its own program/data memory. A program cannot see the other contexts' registers or memory, and cannot tell which context it runs on (context visibility is implicit, PRD 7). All four start at the same RESET_ADDR.

4.1.1 Figure 2.1: What a program sees



Four harts, one shared datapath, one 16 KB bank each

Each hart's state is its own; the datapath exists once and serves the harts in strict rotation, so a hart's timing depends on nothing the other three do. The bank a hart sees is selected by the hart, never by an address bit, which is why four images linked at the same base run side by side.

4.2 ISA

RV32I exactly as kestrel implements it (PRD 3, [REF-7]): every base instruction (unpriv §2.1); FENCE (unpriv §2.1.7) and FENCE.I (Zifencei, unpriv §4.1) retire as no-ops; MRET (priv §3.3.2) falls through to the next instruction. Self-modifying code works without FENCE.I: a store is visible to the context's next fetch, which is the ordering Zifencei exists to provide.

4.3 CSRs

There is no CSR state. Every CSR instruction (Zicsr, unpriv §4.2) reads zero and discards its write, without trapping – the Zicntr counters (`rdcycle`, `rdtime`, `rdinstret`; unpriv §4.3) and `mhartid` (priv §3.1.5) included, so a program cannot tell which context it is (by design, PRD 7). There is no trap vector (`mtvec`, priv §3.1.7, is accepted and ignored), and `WFI` (priv §3.3.3) is illegal. PRD 3.3 lists these departures from the RISC-V specification and why.

4.4 Halts

There are no traps. The exceptions the ISA defines – instruction-address-misaligned (unpriv §2.1.5), load- and store-address-misaligned (unpriv §2.1.6), ECALL and EBREAK (unpriv §2.1.8) and illegal instruction (priv §3.1.15) – stop the context that raises them, and only that context (context-lock, PRD 7):

Table 2.1: Halt causes (kestrel's encoding)

Cause	Event
1	ECALL
2	EBREAK
3	Taken branch, JAL or JALR to an address that is not 4-byte aligned
4	Misaligned load (LH/LHU off a half-word, LW off a word)
5	Misaligned store
0xF	Illegal instruction (including the custom-0 opcode until Xcell is built)

The halting instruction does not write a register or memory. The riscv-tests verdict on a context is its last write to gp (x3) at the ECALL halt: 1 is a pass.

4.5 Timing a program can rely on

None, by design: a context's speed depends on nothing it does. Every instruction – ALU, load, store, taken or not-taken branch – takes exactly four clocks of its context, whatever the other contexts are doing.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Interfaces

5.1 Core memory ports

`gatling_core` has an instruction port and a data port, each carrying the requesting context. Both expect registered-read memory: the data is used exactly one cycle after the request, and nothing waits (there is no wait input).

Table 3.1: gatling_core interfaces

Port	Signals	Contract
Instruction	`imem_ctx`, `imem_addr` -> `imem_rdata`	The word at {ctx, addr} the next cycle
Data	`dmem_req`, `dmem_ctx`, `dmem_addr`, `dmem_wstrb`, `dmem_wdata` -> `dmem_rdata`	Word-aligned address, lane strobes; a load's word the next cycle
Run	`hart_run[NUM_HARTS-1:0]` (four by default)	Per context: 1 run, 0 hold at RESET_ADDR
Status	`halted[3:0]`, `halt_cause[3:0][3:0]`	Per context
Retire	`rvfi_*`, `rvfi_ctx`	riscv-formal RVFI, one beat per cycle at most, tagged with its context

5.2 Board memory map (gatling_mem_bram, over UART -> AXI4-Lite)

Table 3.2: Board memory map

Address	Name	Access
0x0_0000 + 0x4000*c	context c memory (16 KB; addr[16] aliases)	R any time; W only while context c is stopped (dropped otherwise)
0x2_0000	CTRL	W bits[3:0] run per context; R {halted[7:4], run[3:0]}
0x2_0004	BUILD_ID	R “GATL” (0x4741_544C)
0x2_0100 + 0x10*c	STATUS, CYCLES, INSTRET, GP of context c	R; kept after a stop, cleared when the context is started

5.2.1 Figure 3.1: The board address map

Byte address (18 bits)	addr[17]	Selects	Access
0x0_0000 - 0x0_3FFF	0	context 0 memory: bank 0 (16 KB, addr[13:2] = word)	R any time; W only while context 0 is held (run[0] = 0), else dropped with B = OKAY
0x0_4000 - 0x0_7FFF	0	context 1 memory: bank 1	as bank 0, gated by run[1]
0x0_8000 - 0x0_BFFF	0	context 2 memory: bank 2	as bank 0, gated by run[2]
0x0_C000 - 0x0_FFFF	0	context 3 memory: bank 3	as bank 0, gated by run[3]
0x1_0000 - 0x1_FFFF	0	alias of 0x0_0000 - 0x0_FFFF (addr[16] is not decoded)	same as the bank it aliases
0x2_0000	1	CTRL	W: bits[3:0] = run per context (1 release, 0 hold at RESET_ADDR). R: {halted[7:4], run[3:0]}
0x2_0004	1	BUILD_ID	R: "GATL" (0x4741_544C)
0x2_0100 + 0x10*c + 0x0	1	STATUS of context c (addr[5:4] = c)	R: {cause[7:4], 2'b0, halted[1], run[0]}
0x2_0100 + 0x10*c + 0x4	1	CYCLES of context c	R: clocks since its run bit rose, frozen at its halt
0x2_0100 + 0x10*c + 0x8	1	INSTRET of context c	R: its retired instructions (RVFI beats, the trap beat included)
0x2_0100 + 0x10*c + 0xC	1	GP of context c	R: the last value it wrote to x3 (the riscv-tests verdict; 1 = pass)

Lavender: memory banks (block RAM). Blue: registers. Grey: alias. The core never sees the bank field: inside the core a context addresses its whole bank at every address, so the four images are linked at the same base (the riscv-tests battery: 0x8000_0000, which is also the core's RESET_ADDR). STATUS, CYCLES, INSTRET and GP keep their values after a context stops and clear when its run bit rises.

gatling_mem_bram address map over AXI4-Lite: four banks, the alias, CTRL, BUILD_ID and the per-context status block

A host loads a context by stopping it, writing its bank and setting its run bit; the other contexts can keep running throughout. The host driver is `projects/fpga-systems/NexysA7/riscv-ip/gatling/bin/gatling_board.py`.

6 Performance

Table 4.1: Throughput

Measure	Value
Clocks per instruction, per context	4 (exactly; measured CYCLES/INSTRET 4.000)
Instructions per clock, core, four contexts running	1
Taken-branch penalty	0
Load-use penalty	0

The clock and area on the Nexys A7-100T are in the MAS, chapter 7, and the board README.

The shape of the table is the fine-grained multithreading trade as the textbooks state it: throughput across threads is bought with the latency of each thread (H&P6 §3.11; H&H §7.7.7). A context runs at a quarter of the clock and the core retires one instruction per clock. gatling's addition is that the four clocks are exact. Because the context count equals the pipeline depth there is no stall for the rotation to skip, so there is no load-use or taken-branch penalty to hide, only the window (MAS chapter 4; [REF-3]).

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

7 GATLING-RV32ITETRAD Micro-Architecture Specification Index

Version: 0.1 **Date:** 2026-10-22 **Purpose:** Block-level micro-architecture specification for gatling, the four-context barrel RV32I

7.1 Document Organization

Note: All chapters linked below for automated document generation.

7.1.1 Front Matter

- Document Information

7.1.2 Chapter 1: Overview

- Machine Organization

7.1.3 Chapter 2: Functional Blocks

- gatling_core (top)
- gatling_regfile
- gatling_mem_bram

7.1.4 Chapter 3: Internal Interfaces

- The Memory Contract

7.1.5 Chapter 4: Micro-Architecture Behavior

- The Rotation

7.1.6 Chapter 5: Verification

- DV and Formal

7.1.7 Chapter 6: Configuration

- Parameters and Filelists

7.1.8 Chapter 7: Board Result

- Nexys A7-100T

7.2 Related Documentation

- **GATLING HAS** - Hardware Architecture Specification (external view)
- **The Barrel Model** - Why four contexts
- **KESTREL MAS** - The decode, immediate and ALU blocks gatling reuses
- **GATLING PRD** - Requirements and owner decisions
- **Reading list** - The barrel's primary sources (entries 16-20) and the textbook chapters cited

---

Last Updated: 2026-10-22 **Maintained By:** RTL Design Sherpa Project

8 The Barrel Model: Why Four Contexts

Version: 0.1 **Date:** 2026-10-22 **Purpose:** The study chapter behind gatling (PRD 8.1): what kestrel's single cycle costs, why cutting it into stages does not help one instruction stream, and how four interleaved streams make the cut free.

8.1 1. The cost of one cycle

kestrel executes an RV32I instruction in one clock: the program counter, the register-file read, the ALU, the data memory and the register write are one combinational path. That is the teaching point of kestrel and it is also its price. On the Nexys A7 the path that bounds the clock runs from the instruction block RAM, through the register read and the ALU add, into the data memory's address (kestrel MAS, "Board result"):

Table 1.1: kestrel's single cycle on the Nexys A7

kestrel, Nexys A7-100T	Value
Worst path at 100 MHz (IMPL=explore)	11.6 ns
Clock closed	75 MHz (+0.156 ns)
CPI	1.039 (1.00 plus a cycle per load)
Throughput	72.2 MIPS
LUTs	1,899

Every instruction pays for the longest one. A shift, an add and a load all wait the 11.6 ns that a load-address calculation needs.

8.2 2. The obvious fix, and why it does not pay for one stream

Cut the path into stages and clock it faster. Four stages – fetch, decode, execute, write back – each hold a quarter of the work, so the clock could rise toward four times kestrel's.

For one instruction stream the gain mostly evaporates. The next instruction depends on this one: its operands may be this instruction's result (a data hazard), and its address may be this instruction's branch target (a control hazard). A pipelined single-stream core needs forwarding paths, hazard detection, stall logic and branch prediction or flushes to keep the stages busy (P&H §4.8-4.9; H&P6 App. C) – the falcon-suite ladder above kestrel exists to teach exactly that. Without them, the simplest correct multi-cycle machine runs one instruction at a time through all four stages: a CPI of 4 at up to four times the clock, which is roughly where it started.

8.3 3. The interleave

gatling keeps the four stages and removes the dependence instead: the stages are fed from FOUR independent instruction streams, one per hardware context, taken in strict rotation.

This is fine-grained multithreading in the textbook sense – switch threads every cycle, in round-robin order (H&P6 §3.11; P&H §6.4; H&H §7.7.7) – and it is older than the pipelining vocabulary used to describe it. Thornton's CDC 6600 ran ten peripheral processors through one shared execution unit this way in 1964 (papers, entry 16; H&P6 App. M.5); the Denelcor HEP used it to hide pipeline and memory latency with no forwarding and no caches (papers, entries 17 and 18); the Tera MTA pushed it to 128 streams (papers, entry 19); Sun's Niagara brought it back as a four-thread core (papers, entry 20). The textbooks present the interleave as a way to *hide* stalls. gatling uses it to *remove* hazards, which is what happens when the number of streams is fixed equal to the number of stages:

```
cycle     t     t+1   t+2   t+3   t+4   t+5
F        c0    c1    c2    c3    c0    c1
D              c0    c1    c2    c3    c0
X                    c0    c1    c2    c3
W                          c0    c1    c2
```

Every cycle each stage holds a different context. Context c0's next instruction is fetched at t+4, after its current one has written its PC (end of X, t+2) and its register (end of W, t+3). So:

- no data hazard – a context never reads a register that an instruction of its own still in flight is about to write;
- no control hazard – a branch resolves before that context's next fetch, so there is nothing to predict and nothing to flush;
- no forwarding, no stalls, no interlocks for the base ISA.

Each context still takes four cycles per instruction (CPI 4), but four of them retire one instruction each per four cycles: the core retires one instruction per clock. The multi-cycle cut costs nothing when there are four streams to fill it – the kestrel to merlin lesson inverted: instead of pipeline stages hiding latency for one stream, contexts hide it for each other.

8.4 4. What the four stages buy in memory

Each stage is one block-RAM latency long, and that is not a coincidence the design has to work around; it is what the design uses:

Table 4.1: every memory latency is exactly one stage

Memory	Address presented	Data used
Instruction RAM	F	D
Register file (block RAM)	D	X
Data RAM (load)	X	W

kestrel has to hold the core for a cycle on every load (`mem_wait`) because its block RAM answers a cycle late. gatling has no wait signal at all.

8.5 5. What it costs

- Four PCs and four register files instead of one – the “cheap replication” (PRD 4.1). Harris and Harris make the same point: multithreading “replicates only the PC and register file, not the execution units and memories” (H&H §7.7.7). The register files are one 128-entry block RAM (two copies for two read ports), not 4,096 flops.
- One datapath: decode, immediates and ALU are kestrel’s, unchanged.
- A context is a quarter of the machine. A single program runs at a quarter of the clock rate – the fine-grained trade as Hennessy and Patterson state it, multithreaded throughput bought with single-thread latency (H&P6 §3.11), and what Jordan measured on the HEP (papers, entry 18). The barrel pays off only when there are four things to do – which, for a tetrad of four cells, is the point.

8.6 6. Measured

On the same Nexys A7-100T (gatling board README, “Board result”; MAS chapter 7):

Table 6.1: The single cycle against the barrel

	kestrel	gatling
Worst path	11.6 ns (the whole instruction)	9.4 ns (the X stage)
Clock	75 MHz	100 MHz
Programs at once	1	4
CPI per program	1.039	4.004
Peak throughput	72.2 MIPS	100 MIPS
LUTs	1,899	2,281

Four harts cost 20% more logic and run 39% more instructions per second in total, with every instruction taking the same four clocks. The clock did not rise fourfold: the X stage – register read through ALU or branch to the next PC – is still most of kestrel’s instruction, and routing is half of the worst path (52%) on this part. The remaining stages have slack to give, which is where a deeper rotation would go, as the HEP’s eight-stage pipeline and Tera’s 128 streams did (papers, entries 17 and 19).

8.7 7. Sources

Citations follow the suite's conventions (kestrel study book, chapter 1, "Conventions"): the ISA by section of the pinned manual, papers by entry in `references/microarchitecture-papers.md`, textbooks by edition and section.

Table 7.1: Sources cited in this chapter

Key	Source
H&P6	J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019. §3.11 "Multithreading: Exploiting Thread-Level Parallelism to Improve Uniprocessor Throughput"; Appendix C "Pipelining: Basic and Intermediate Concepts"; Appendix M.5 "The Development of Pipelining and Instruction-Level Parallelism"
P&H	D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2021. §4.8 "Data Hazards: Forwarding versus Stalling"; §4.9 "Control Hazards"; §6.4 "Hardware Multithreading"
H&H	S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022. §7.7.7 "Multithreading"
papers, entries 16-20	Thornton 1964 (CDC 6600); Smith 1978 and Jordan 1983 (HEP); Alverson et al. 1992 (Tera MTA); Kongetira et al. 2005 (Niagara)
kestrel MAS	Chapter 7, "Board result": the kestrel column of Tables 1.1 and 6.1

9 GATLING PRD — Tetrad Compute Engine

Version: 0.4 (2026-10-21 — the deliberate departures from the RISC-V spec recorded in 3.3; no Zicntr.) **Date:** 2026-10-21 **Owner decision inputs:** “Instructions” (over MMIO) for fabric handshake; mini RV per tetrad (over per-cell ALU+FSM); codename gatling (barrel machine). **Supersedes:** PRD 0.1 wording that described gatling as replacing the per-cell ALU — the ALU stays; gatling replaces the execution FSM as its driver.

9.1 1. Purpose

Give each tetrad of the compute tile a programmable engine small enough to instantiate four times per tile, powerful enough to run compiled programs over cell input data, and honest enough to teach: the barrel processor as the kestrel→merlin lesson, inverted — instead of pipeline stages hiding latency for one stream, N contexts hide the multi-cycle latency of one shared datapath.

Owner ruling 2026-10-19: gatling sequences the four ALUs of its adjacent cells. The barrel RV32I core is the programmable sequencer for its tetrad — each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. *The program is the config*. Mode bits that remain (element width, credit batch N) move to the CFG read path. Gatling is also a barrel RISC-V with custom commands (Xcell: CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT) — the program is the schedule. Gatling never originates config traffic; cell configuration arrives in-band over delta.

9.2 2. Non-goals

- Not a falcon-suite rung. No performance chase; IPC-per-context ~ 1 is the target, not a benchmark result.
- Not per-cell processors. One engine per tetrad; the four cells are the four contexts.
- No caches, no virtual memory, no interrupts/CLINT, no M/A/F. (Machine state is 4× {PC, x1..x31} + mode CSR shadow. Full stop.)
- No CSRs as a subsystem — `Zicsr` deliberately absent; config reaches the program through the Xcell `CFG` instruction. Every CSR access is kestrel's stub (3.3), the Zicntr counters included.

9.3 3. ISA — RV32I + Xcell

Base RV32I exactly as kestrel implements it (same opcode map, same trap/halt contract) so the spike lockstep flow and rv32ui battery carry unchanged. The Xcell extension occupies the custom-0 opcode (`0001011`, 0x0B) — reserved space, zero collision risk with future standard extensions.

9.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)

```
 31                      15 14   12 11      7 6       0
+-------------------------+-------+---------+--------+
|        imm[31:12]       | funct3|   rd    | 0x0B   |  CFG
|        000000000000     |  000  |   rd    | 0x0B   |  POP
|        000000000000     |  001  |  rs1    | 0x0B   |  PUSH
|        000000000000     |  010  |  00000  | 0x0B   |  ECTX
|        000000000000     |  011  |  rs1    | 0x0B   |  FLUSH
|        000000000000     |  100  |  rs1    | 0x0B   |  KICK
|        000000000000     |  101  |  rs1    | 0x0B   |  WAIT
+-------------------------+-------+---------+--------+
funct3 110/111 : reserved.
funct3 010 (ECTX) and 011/100/101 (FLUSH/KICK/WAIT) added in v0.2.
The v0.1 WAITCD/WAITGT placeholders are superseded by KICK/WAIT
pending
the tetrad MAS.
```

Instr	Action	Blocking behavior
`CFG rd, imm`	rd ← mode-register[imm]: 0=element mode (0=bf16: 2 bf16 lanes per 32-bit line, fp32 accumulate; other values reserved for the int8/fp32 roadmap per alu-infer PRD `func_sel[8:7]`), 1=credit batch N, 2=context ID (read-only, see §7 open item), 3+=reserved	never blocks
`POP rd`	rd ← next element of this context's input memory; head advances; consumed-line credit returned to rapids per credit-batch config	context stalls if input empty — hardware interlock, not a spin

Instr	Action	Blocking behavior
`PUSH rs1`	rs1 → this context's result memory; tail advances; grant requested on result path; packetizer batches X flits	context stalls if no grant
`ECTX`	context done: freeze PC, raise done status to tetrad MonBus, release datapath share; context restarts on reset/config reload	drains at next context boundary
`FLUSH rs1`	rs1[14:0] → cell `FLUSH_DEST` {ext,level,X,Y,Z}; rs1[18:15] → cell `FLUSH_TAG` (RAPIDS sink-channel tag); trigger cell flush-and-send of result memory	context stalls until drain controller accepts command
`KICK rs1`	raise tetrad coordination event on the tetrad MonBus with value `rs1`; provisional encoding pending tetrad MAS	non-blocking once event is posted
`WAIT rs1`	stall context until the tetrad coordination condition in `rs1` is observed (e.g. group-done bitmask); provisional encoding pending tetrad MAS	context stalls until condition true

Design note (owner doctrine): fixed hardware owns the line-rate handshake (credit/grant detection, memory ports); the program owns policy. POP/PUSH blocking interlocks subsume most waiting — a context that would wait simply isn't scheduled. WAIT is an explicit escape hatch for cross-context coordination, not the common case.

Element semantics: with element width < 32, POP zero-extends into rd and PUSH takes the low bits of rs1; packing/unpacking is the program's business (one ANDI/SLTI pair, honest and visible).

9.3.2 3.2 Assembly mnemonics

`gatling_pop`, `gatling_push`, `gatling_cfg`, `gatling_ectx`, `gatling_flush`, `gatling_kick`, `gatling_wait` as canonical toolchain names; assembler aliases `pop/push/cfg/ectx/flush/kick/wait` live in the Xcell binutils fragment when the flow arrives (§8).

9.3.3 3.3 CSRs, and the deliberate departures from the RISC-V spec

There is no CSR state. Zicntr (`cycle`, `time`, `instret`) is not implemented: the counters were built and then removed (owner 2026-10-21) – no gatling program reads them, and they cost about 344 LUTs per core. A program that needs a time base reads one from its memory map.

Deliberate departures (owner 2026-10-21, after a review against the RISC-V books in the library – Patterson & Hennessy, Harris & Harris, Ledin, Zhang):

Departure	The spec	Why gatling differs
`mhartid` reads 0 on every context	hart IDs are unique	contexts are implicit (7, item 2): no program can tell which cell it runs
Every CSR reads 0 and drops writes, the counters included	an unsupported CSR access is an illegal instruction	kestrel's stub, kept so the rv32ui p-env preamble (`csrw mtvec`, …) runs
No trap vector: a trap halts the context (4.4); MRET is a no-op	M-mode with `mtvec`/`mepc`/`mcause` is mandatory	context-lock is the halt contract; a tetrad is not a CPU
`WFI` is illegal and halts the context	`WFI` may be a no-op	waiting belongs to the Xcell `WAIT` (TASK-002), which has something to wait on
The rotation never skips a halted or held context	fine-grained multithreading usually skips stalled threads (H&P 3.11, P&H 6.4)	skipping would let a context's next instruction issue sooner than four cycles behind its last and bring back the hazard logic the barrel exists to remove; a tetrad runs all four

Departure	The spec	Why gatling differs
		cells

9.4 4. Barrel microarchitecture

9.4.1 4.1 The model

Kestrel's single cycle is PC + regfile-read + ALU + memory in one combinational path — the teaching point and the timing/area cost. Gatling splits that into a 4-state FSM:

```
F  : address = ctx[sel].pc; fetch word from ctx program image
D  : decode; read ctx[sel] regfile (rs1/rs2 by decoded index)
X  : ALU / Xcell unit (POP/PUSH/CFG execute here; interlock check)
W  : writeback to ctx[sel] regfile; pc advance (or branch target)
```

Each FSM state advances context `sel` one step; `sel` rotates 0→1→2→3→0. Steady state: every context retires one instruction per 4-cycle window ⇒ 1 IPC per context, one gatling ALU, one decoder, one Xcell unit, **four** register files + PCs (the cheap replication) — replacing the cell PRD's four execution FSMs while each cell keeps its own bf16 ALU datapath (owner ruling 2026-10-19).

9.4.2 4.2 Structural hazards (all resolved by rotation, by construction)

- ALU/decoder/memory-port conflict: impossible — one user per state.
- Input/result memory port conflict across contexts: impossible — each context owns its cell's memories, addressed by `sel`.
- Interlock stall (POP empty / PUSH no-grant): only `sel` stalls; rotation skips it (bubble) — three contexts keep the datapath busy.

9.4.3 4.3 Branch behavior

Resolved in X (kestrel semantics carry: the branch decision is one comparison) and written to the context's PC at the end of X, two cycles before that context's next fetch. As built (2026-10-21) a taken branch therefore costs nothing – no bubble, no flush machinery, no BTB, no prediction. The predictor study belongs to peregrine, not here.

9.4.4 4.4 Halt/trap

No traps (no exceptions in base scope; illegal instruction → context-local lock with status bit via MonBus, other contexts continue — a tetrad is not a CPU, one bad cell must not sink the tile). `ECTX` is the normal exit. Reset is per-tile from the tile root; program load via the fabric config path (PKT_CONFIG) into each cell's program image region — kestrel's `mem_loader`/AXIL pattern becomes the in-band loader, per the standing in-band-only fabric decision.

9.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)

Cell PRD block	Stand / change
delta_mesh_cell node, containment	stands — gatling sits behind the node's local port
Input memory, line fill by node from rapids	stands; per-context, addressed by `sel`
Result memory, X-flit batching, packetizer, grants	stands
Consumed-line credits to rapids, batch N (default 1)	stands; N read via CFG
Element widths B4/8/16/32	superseded — alu-infer's bf16 element model (2 bf16 lanes per 32-bit line, fp32 accumulate) is normative; CFG[0] reports the alu-infer element mode (bf16 today; int8/fp32 modes are a roadmap item in the alu-infer PRD via `func_sel[8:7]`)
ALU + execution FSM (config-selected fn)	**replaced by gatling as the execution FSM driver** — the cell's bf16 ALU datapath remains; gatling issues it via POP/PUSH and the alu-infer elastic valid/ready seam (owner ruling 2026-10-19)
MonBus observation (per-cell → per-tetrad merge)	stands; gatling adds per-context status (active/stall-reason/done) to the observed set

Consequences that must not fork: credit counting is in *elements consumed* (POP), not instructions; grants are in *elements pushed* (PUSH), not flits — flit batching stays inside the packetizer.

9.6 6. Verification

- **Base ISA:** spike lockstep on RV32I exactly as kestrel does it — reuse the rds-dv flow; rv32ui battery is the gate (42/42 precedent).
- **Xcell unit:** directed cocotb — POP empty-stall / credit batch N=1,2,4; PUSH grant/backpressure; CFG read of mode shadow; ECTX done/observability; element-width zero-extend and slice.
- **Barrel properties:** cocotb assertions — one retirement per context per rotation window; stall of context k never stalls j≠k; rotation fairness (round-robin, no starvation with all-runnable).
- **Tetrad integration:** the eventual `tetrad.sv` TB drives four input-memory streams, checks per-context result memories and the credit/grant ledger against the cell PRD reference model.
- **Formal (where it pays):** deadlock freedom of the interlock (POP-empty ∨ PUSH-no-grant ∧ no ECTX ⇒ eventually credit ∨ grant), following the repo's targeted-formal doctrine.

9.7 7. Open items (owner decisions needed)

1. **ALU-infer relationship:** RESOLVED 2026-10-19 — gatling is the programmable sequencer for the tetrad's four bf16 ALUs; each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. The v0.1 wording that made gatling "the ALU
   - execution FSM" is superseded.
2. **Context visibility:** RESOLVED 2026-10-21 (owner) — implicit. Software cannot tell the contexts apart; binaries are uniform. CFG[2] stays reserved.
3. **Program image placement:** RESOLVED 2026-10-21 (owner) — separate. Each context has its own program/data region (gatling_mem_bram: one 16 KB bank per context, seen at every address); the cell's POP input memory is separate and arrives with Xcell.
4. **Illegal-instruction policy:** RESOLVED 2026-10-21 (owner) — context-lock. Any halt (ecall, ebreak, illegal, misaligned target or access) locks only its context; the other three keep running.
5. **Xcell binutils/toolchain:** hand-assembled hex for bring-up; gcc/binutils fragment later. Non-blocking.

Resolved/added in v0.2: the FLUSH/KICK/WAIT Xcell encodings of §3.1 (answer TILE_SPEC §11.14 and the alu-infer PRD §5 tetrad-coordination pair). Their exact field layouts and MonBus integration remain provisional pending the tetrad MAS.

Scope of the first RTL (owner, 2026-10-21): the RV32I barrel first – four harts running the rv32ui battery on the Nexys A7 – with the Xcell custom-0 opcode decoding as illegal (context-lock) until the cell seam exists. Xcell lands in the next pass.

9.8 8. Deliverables & sequencing

1. `docs/barrel_model/` study chapter (kestrel cost → interleave fix) — writes first, it is the justification.
2. `gatling_has` + ISA appendix (Xcell table as authority).
3. RTL: fubs (kestrel-derived) → `gatling_top` barrel + Xcell unit → flists; unit TBs per fub on repo rails.
4. Spike lockstep bring-up of the RV32I subset; then Xcell directeds.
5. Tetrad integration when `tetrad.sv` lands in `tile/`.

9.9 9. History

- 2026-10-21: PRD 0.4. The deliberate departures from the RISC-V spec recorded (3.3): mhartid 0, the CSR stub, no trap vector, WFI illegal, a strict rotation. Zicntr counters were added and then removed the same day (owner): unused by any program, ~344 LUTs.
- 2026-10-21: PRD 0.3. First RTL (gatling_core, gatling_regfile, gatling_mem_bram): the RV32I barrel with kestrel's decode/imm_gen/ALU unchanged, every memory a one-stage block-RAM latency, no wait signal; four harts on the Nexys A7. Owner decisions: PRD 7 items 2-4 (implicit context, separate program regions, context-lock); RV32I barrel before Xcell. 4.3 corrected: a taken branch costs no bubble.
- 2026-10-19: PRD 0.2. Owner ruling: gatling sequences the four ALUs of its adjacent cells — the barrel core is the programmable sequencer for the tetrad, the cell's bf16 ALU remains the datapath, and gatling never originates config traffic. Added FLUSH/KICK/WAIT Xcell encodings; fixed POP to read `rd`; updated CFG[0] to report the alu-infer element mode.
- 2026-10-17: PRD 0.1 scaffold. Owner decisions: instructions over MMIO; mini RV per tetrad; codename gatling; four contexts = four cells; blocking POP/PUSH interlock; ECTX exit. Cell PRD 0.1 contracts confirmed unchanged outside the replaced ALU/FSM block.

10 Microarchitecture Reading List

The research literature behind the falcon suite's design decisions. Each entry names the decision it guides and the rung(s) it serves; cite these in rung books and RTL comments the same way the ISA manual is cited (by section/idea, not page). Reading order follows the ladder — foundations first, OOO last; the barrel (gatling) sits beside the ladder, so its sources come after.

Availability is noted honestly: most IEEE/ACM papers are paywalled, but nearly all have free author or institutional mirrors worth finding.

10.1 Foundations — kestrel & merlin

1. **D. Patterson and C. Séquin, "RISC I: A Reduced Instruction Set VLSI Computer," ISCA 1981.** *Guides:* kestrel's whole shape — the case for a register-file-centric datapath, single-cycle simplicity, and why the ISA should map 1:1 to hardware. The original "see everything" machine.

2. **M. Flynn, "Very High-Speed Computing Systems," Proceedings of the IEEE, 1966.** *Guides:* pipelining as a concept (the Flynn taxonomy ride-along); merlin's raison d'être — overlapping instruction stages for throughput rather than raw clock.

3. **J. Hennessy, N. Jouppi, et al., "MIPS: A Microprocessor Architecture," MICRO 1982.** *Guides:* merlin's stage partition (IF/ID/EX/MEM/WB) — the paper that established the five-stage RISC pipeline everyone still teaches.

10.2 Prediction, traps, caches — peregrine

4. **J. E. Smith, "A Study of Branch Prediction Strategies," ISCA 1981.** *Guides:* peregrine's dynamic prediction choice — the original 1-bit counter and its aliasing/measurement methodology; the baseline our 2-bit bimodal improves on.

5. **J. K. F. Lee and A. J. Smith, "Branch Prediction Strategies and Branch Target Buffer Design," IEEE Computer, 1984.** *Guides:* the BTB half of peregrine's frontend — sizing, organization, and the interaction between target caching and direction prediction.

6. **J. E. Smith and A. R. Pleszkun, "Implementing Precise Interrupts in Pipelined Processors," IEEE Transactions on Computers 37(5), May 1988, 562–573.** *Guides:* peregrine's precise-exception strategy — the classic taxonomy (freeze/history-buffer/future-file/reorder-buffer); the reason a simple in-order pipeline can present architecturally exact trap state.

7. **N. Jouppi, "Cache Write Policies and Performance," ISCA 1993.** *Guides:* peregrine's write-back-with-dirty-bit D$ decision — the write-through escape hatch in the spec exists because this paper quantifies when each policy wins.

10.3 Out-of-order — gyrfalcon

8. **R. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," IBM Journal of Research and Development 11(1), 1967, 25–33.** (freely available from IBM) *Guides:* gyrfalcon's reservation stations and common data bus — the original OOO. Note what it *lacks* (precise exceptions), which sets up the next two papers.

9. **W. Hwu and Y. Patt, "Checkpoint Repair for High-Performance Out-of-Order Execution Machines," IEEE Transactions on Computers 36(12), 1987, 1496–1514.** *Guides:* gyrfalcon's chosen mispredict recovery — checkpoint the rename state at each branch dispatch and restore on flush, instead of undoing history entry by entry. Our default rollback mechanism.

10. **V. Popescu, M. Schultz, J. Spracklen, et al., "The Metaflow Architecture," IEEE Micro, 1991.** *Guides:* speculative OOO with register renaming as a complete machine — the conceptual ancestor of every P6-style design, and the cleanest statement of "dispatch speculatively, commit in order."

11. **K. Yeager, "The MIPS R10000 Superscalar Microprocessor," IEEE Micro 16(2), April 1996, 28–41.** (free mirrors widely available) *Guides:* gyrfalcon's explicit rename map + active-list organization and branch checkpointing — the closest single reference for our rung 4 design. Also the acknowledged inspiration for BOOM.

12. **R. Kessler, "The Alpha 21264 Microprocessor," IEEE Micro, March–April 1999.** *Guides:* an industrial-strength OOO RISC — clustered scheduling, the register file port problem, and what happens when gyrfalcon's teaching simplifications are removed. Read last.

13. **J. E. Smith and G. Sohi, "The Microarchitecture of Superscalar Processors," Proceedings of the IEEE 83(12), December 1995.** *Guides:* the survey that ties 8–12 together — use it as gyrfalcon's chapter skeleton and as the checklist that no OOO concept was silently dropped.

10.4 ISA rationale & prior art — all rungs

14. **A. Waterman, “Design of the RISC-V Instruction Set Architecture,” PhD dissertation, UC Berkeley, 2016.** (free) *Guides:* why RV32I looks the way it does — the ISA-design rationale behind kestrel’s decode table, and the cheapest way to answer “why does the spec do X?” without reading all 909 pages.

15. **C. Celio, “The Berkeley Out-of-Order Machine (BOOM),” UC Berkeley technical report, 2015.** (free) *Guides:* prior art and differentiation — the open-source OOO RISC-V baseline. gyrfalcon is deliberately *not* BOOM: SystemVerilog rather than Chisel, small enough to hold in your head, checkpoints rather than a unified physical register file. Know it so we don’t accidentally reimplement its complexity.

10.5 Interleaved multithreading — gatling

16. **J. E. Thornton, “Parallel Operation in the Control Data 6600,” Proc. AFIPS Fall Joint Computer Conference, Part II, 1964, 33–40.** *Guides:* the barrel itself. The 6600’s ten peripheral processors shared one execution unit in strict rotation, each taking a slot every ten minor cycles, so no peripheral processor ever had two instructions in flight. gatling’s four-context rotation is that organization at RV32I (barrel model §3). Thornton’s 1970 book *Design of a Computer: The Control Data 6600* (Scott, Foresman) is the long form.

17. **B. J. Smith, “A Pipelined, Shared Resource MIMD Computer,” Proc. International Conference on Parallel Processing (ICPP), 1978, 6–8.** *Guides:* the Denelcor HEP: fine-grained interleaving used to hide pipeline latency with no forwarding and no interlocks between a process’s own instructions, because a process could issue only once its previous instruction had left the pipeline. gatling’s “one instruction in flight per context” invariant (MAS 1.1) is the same rule, fixed by construction instead of by a queue.

18. **H. F. Jordan, “Performance Measurements on HEP — A Pipelined MIMD Computer,” Proc. 10th International Symposium on Computer Architecture (ISCA), 1983, 207–212.** *Guides:* the measurement side of the trade: single-process speed is the cost of the interleave, and throughput arrives only as the process count fills the pipeline. Barrel model §5 and §6 make the same argument with kestrel’s and gatling’s board numbers.

19. **G. Alverson, R. Alverson, D. Callahan, B. Koblenz, A. Porterfield and B. Smith, “Exploiting Heterogeneous Parallelism on a Multithreaded Multiprocessor,” Proc. Supercomputing ’92, 1992, 188–197.** *Guides:* the Tera MTA, the HEP idea taken to 128 streams per processor to cover memory latency without caches. Where gatling stops (four contexts, one block-RAM latency per stage) is read against where Tera went.

20. **P. Kongetira, K. Aingaran and K. Olukotun, “Niagara: A 32-Way Multithreaded Sparc Processor,” IEEE Micro 25(2), March–April 2005, 21–29.** *Guides:* the modern form: four-thread fine-grained cores with round-robin issue that skips a stalled thread. That skip is the textbook norm and the one thing gatling’s rotation deliberately does not do (PRD 3.3; MAS chapter 4).

10.6 Textbooks

The owner's textbook library (machine-local, one Markdown file per section; `vault/handbook/book-library.md`) covers the same ground at textbook depth. Cite by edition, chapter and section, never by page, with the short key in the second column. The chapters the rung books lean on:

Textbooks cited by the rung books

Book	Key	Where it guides
J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019	H&P6 §N	§3.11 fine-grained, coarse-grained and simultaneous multithreading (gatling); Appendix C pipelining and hazards (merlin; gatling's "why not one stream"); Appendix M.5 the lineage from the 6600 I/O processors through HEP, Tera and Niagara (gatling)
D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2021	P&H §N	§4.8-4.9 data and control hazards (kestrel to merlin); §6.4 hardware multithreading (gatling)
S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022	H&H §N	ch. 6 the RV32I architecture and ch. 7 the single-cycle and pipelined RISC-V microarchitectures (kestrel, merlin); §7.7.7 multithreading as "replicate only the PC and register file" (gatling)
J. Ledin, *Modern Computer Architecture*	Ledin §N	§14.3 simultaneous multithreading, the

Book	Key	Where it guides
and Organization, 2nd ed., Packt, 2022		contrast case for gatling's interleave; ch. 17 the RISC-V ISA
Z. Zhang, *RISC-V Architecture and DSP Processor Design*, Packt, 2026	Zhang §N	ch. 6 arithmetic units (the ALU rungs); a second RISC-V core walk-through to read kestrel against